

SQL

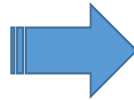
Parte II: Operaciones de conjunto,
Funciones de agregación,
Group by,
Having,

Operaciones de conjunto en SQL

Compatibilidad de Unión
Igual grado y dominio
columna a columna

La operación Unión

R		S	



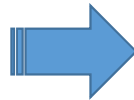
R ∪ S	

Tuplas duplicadas o usar DISTINCT

- ✓ Asociativa
- ✓ Conmutativa

La operación Intersección

R		S	



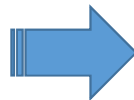
$R \cap S$	

Tuplas duplicadas o usar DISTINCT

- ✓ Asociativa
- ✓ Conmutativa

La operación Diferencia

R		S	



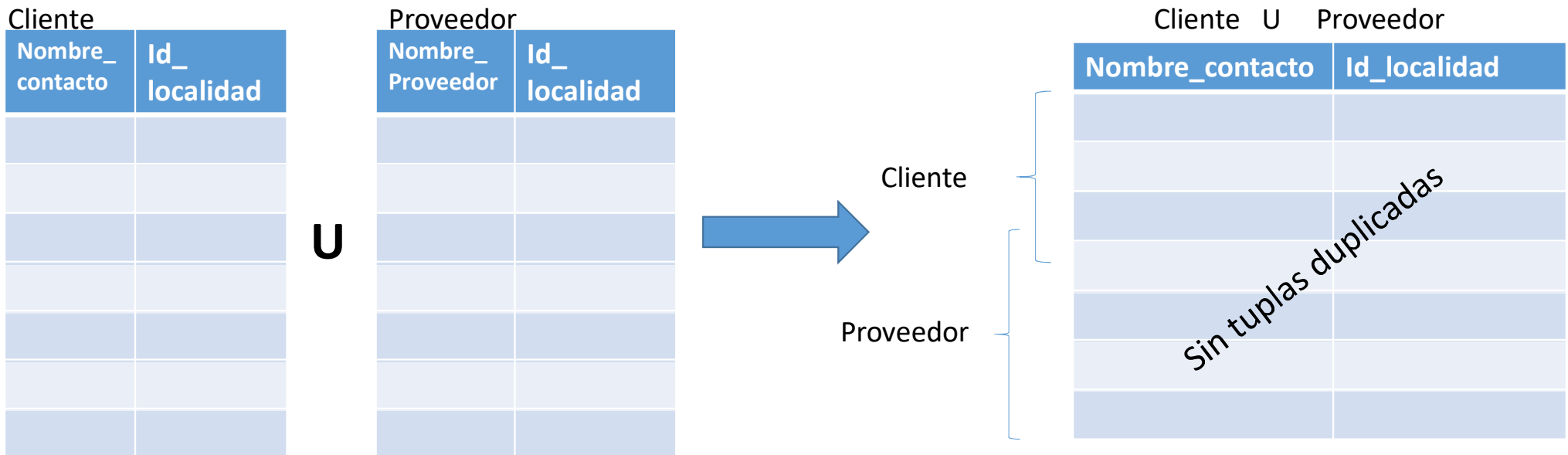
R - S	

Tuplas duplicadas o usar DISTINCT

- ✓ No Conmutativa

Operaciones de conjunto en SQL

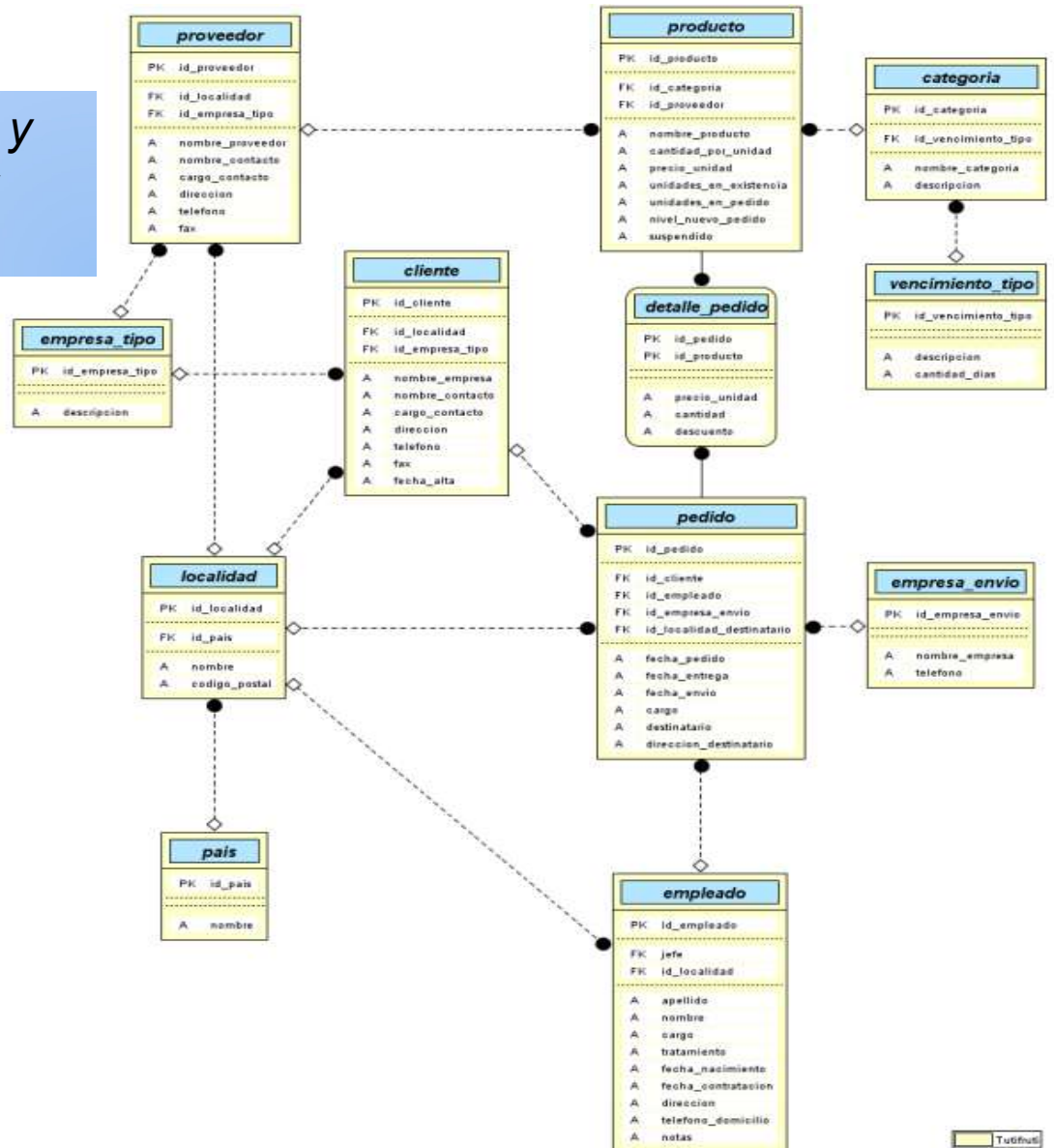
En un listado único de *nombre de contacto* y *id_localidad de residencia*, *juntar clientes y proveedores*



En las operaciones de conjunto a diferencia del producto cartesiano o la reunión, las tuplas de la segunda relación se colocan debajo de las tuplas de la primera relación, no a la par. Por ello, las operaciones de conjunto dan como resultado una nueva relación que tiene el mismo grado que las relaciones que intervienen y cada columna toma el nombre de las columnas de la primera relación en la operación de conjunto.

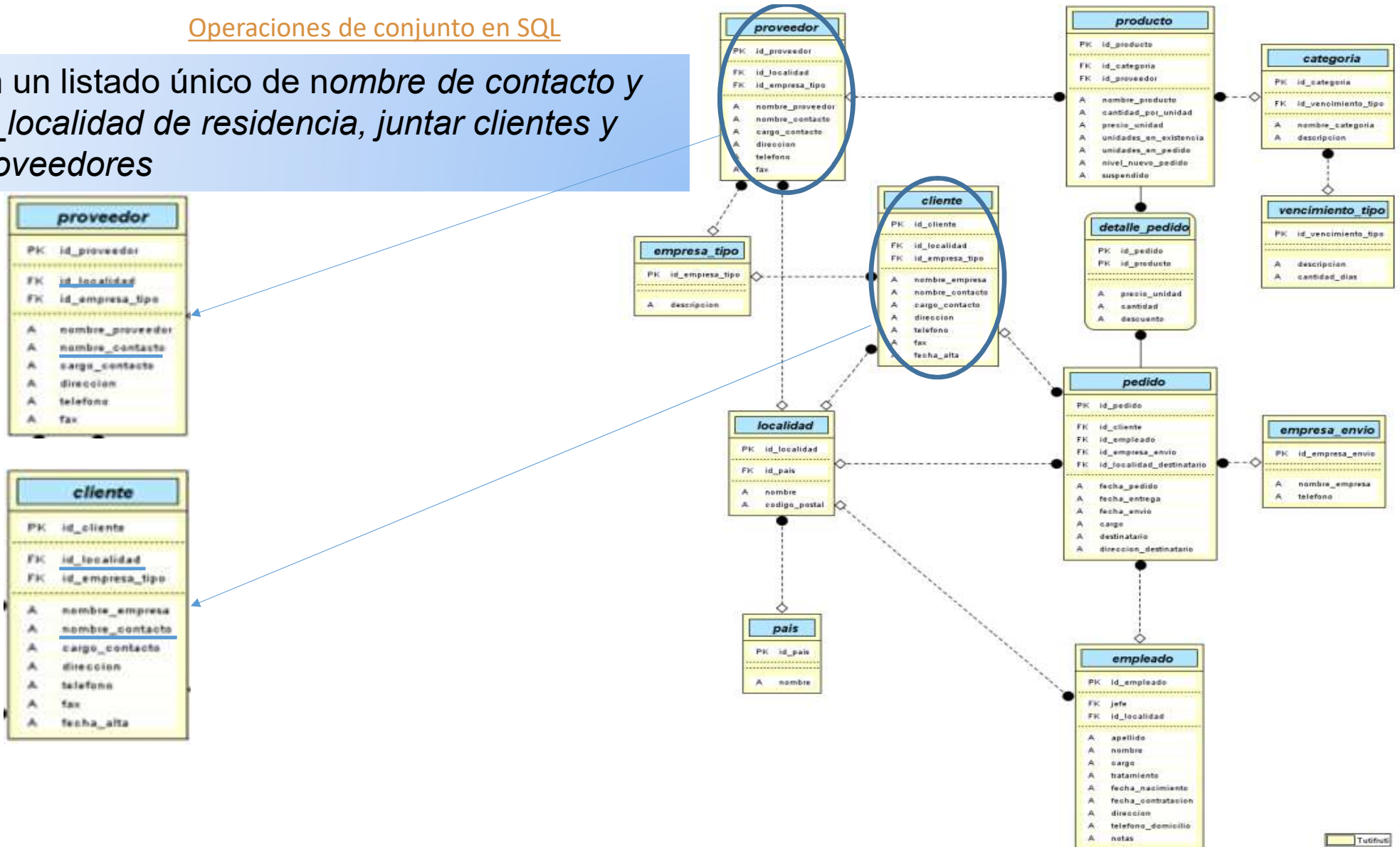
Operaciones de conjunto en SQL

En un listado único de *nombre de contacto* y *id_localidad de residencia*, juntar clientes y proveedores



Operaciones de conjunto en SQL

En un listado único de *nombre de contacto* y *id_localidad de residencia*, juntar clientes y proveedores



Operaciones de conjunto en SQL

En un listado único de *nombre de contacto* y *id_localidad de residencia*, juntar clientes y proveedores

cliente	
PK	id_cliente
FK	<u>id_localidad</u>
FK	id_empresa_tipo
A	nombre_empresa
A	<u>nombre_contacto</u>
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select nombre_contacto, localidad
from main.cliente)

proveedor	
PK	id_proveedor
FK	<u>id_localidad</u>
FK	id_empresa_tipo
A	nombre_proveedor
A	<u>nombre_contacto</u>
A	cargo_contacto
A	direccion
A	telefono
A	fax

Operaciones de conjunto en SQL

En un listado único de *nombre de contacto* y *id_localidad de residencia*, juntar clientes y proveedores

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select nombre_contacto, id_localidad
from main.cliente)

```
1 select nombre_contacto, id_localidad
2 from main.cliente
```

Data Output Explain Messages Notifications

	nombre_contacto character varying (30)	id_localidad integer
1	Sergio Gutiérrez	24
2	Yvonne Moncada	24
3	Patricio Simpson	24
4	Miguel Angel Paolino	88
5	Pascale Cartrain	28
6	Patricia McKenna	30
7	Helen Bennett	31
8	Yoshi Latimer	34
9	Alexander Feuer	53
10	Isabel de Castro	57
11	Roland Mendel	44
12	Felipe Izquierdo	46
13	Helvetius Nagy	48
14	Henriette Pfalzheim	50
15	Antonio Moreno	87
16	Jaime Yorres	121
17	Pedro Afonso	125
18	Jose Pavarotti	17
19	Maria Larsson	19

Operaciones de conjunto en SQL

En un listado único de *nombre de contacto* y *id_localidad de residencia*, juntar clientes y proveedores

(select nombre_contacto, id_localidad
from main.cliente)

(select nombre_contacto, id_localidad
from main.proveedor)

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

```
1 select nombre_contacto, id_localidad
2 from main.cliente
```

Data Output Explain Messages Notifications

	nombre_contacto character varying (30)	id_localidad integer
1	Sergio Gutiérrez	24
2	Yvonne Moncada	24
3	Patricio Simpson	24
4	Miguel Angel Paolino	88
5	Pascale Cartrain	28
6	Patricia McKenna	30
7	Helen Bennett	31
8	Yoshi Latimer	34
9	Alexander Feuer	53
10	Isabel de Castro	57
11	Roland Mendel	44
12	Felipe Izquierdo	46
13	Helvetius Nagy	48
14	Henriette Pfalzheim	50
15	Antonio Moreno	87
16	Jaime Yorres	121
17	Pedro Afonso	125
18	Jose Pavarotti	17
19	Maria Larsson	19

```
1 select nombre_contacto, id_localidad
2 from main.proveedor
```

Data Output Explain Messages Notifications

	nombre_contacto character varying (30)	id_localidad integer
1	Carlos Diaz	129
2	Chandra Leka	135
3	Regina Murphy	5
4	Chantal Goulet	137
5	Antonio del Valle Saavedra	98
6	Mayumi Ohno	96
7	Ian Devling	84
8	Peter Wilson	80
9	Lars Peterson	43
10	Michael Björn	138
11	Petra Winkler	13
12	Martin Bein	39
13	Sven Petersen	33
14	Elio Rossi	105
15	Beate Vileid	120
16	Cheryl Saylor	11
17	Wendy Mackenzie	141
18	Guyliène Nodier	100
19	Robb Merchant	18

Operaciones de conjunto en SQL UNION

En un listado único de *nombre de contacto* y *id_localidad de residencia*, juntar clientes y proveedores

(select nombre_contacto, id_localidad
from main.cliente)

(select nombre_contacto, id_localidad
from main.proveedor)

```
1 select nombre_contacto, id_localidad
2 from main.cliente
```

	nombre_contacto	id_localidad
	character varying (30)	integer
1	Sergio Gutiérrez	24
2	Yvonne Moncada	24
3	Patricio Simpson	24
4	Miguel Angel Paolino	88
5	Pascale Cartrain	28
6	Patricia McKenna	30
7	Helen Bennett	31
8	Yoshi Latimer	34
9	Alexander Feuer	53
10	Isabel de Castro	57
11	Roland Mendel	44
12	Felipe Izquierdo	4
13	Helvetius Nagy	3
14	Henriette Pfalzner	50
15	Antonio Moreno	121
16	Jaime Yorres	125
17	Pedro Afonso	17
18	Jose Pavarotti	19
19	Maria Larsson	19

```
1 select nombre_contacto, id_localidad
2 from main.proveedor
```

	nombre_contacto	id_localidad
	character varying (30)	integer
1	Carlos Diaz	129
2	Chandra Leka	135
3	Regina Murphy	5
4	Cheryl Boulet	137
5	Antonio del Valle Saavedra	98
6	Meximiliano Ohno	96
7	Sam Devlin	84
8	Peter Wilson	80
9	Lars Peterson	43
10	Michael Björn	138
11	Petra Winkler	13
12	Martin Bein	39
13	Sven Petersen	33
14	Elio Rossi	105
15	Beate Vileid	120
16	Cheryl Saylor	11
17	Wendy Mackenzie	141
18	Guyliène Nodier	100
19	Robb Merchant	18

Operaciones de conjunto en SQL UNION

En un listado único de nombre de contacto y id_localidad de residencia, juntar clientes y proveedores

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select nombre_contacto, id_localidad
from main.cliente)

UNION

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select nombre_contacto, id_localidad
from main.proveedor)

```
1 select nombre_contacto, id_localidad
2 from main.cliente
3 union
4 select nombre_contacto, id_localidad
5 from main.proveedor
```

	Data Output	Explain	Messages	Notifications
	nombre_contacto Character varying (30)		id_localidad integer	
1	Michael Holz		42	
2	Wendy Mackenzie		141	
3	Antonio Moreno		87	
4	Matti Karttunen		45	
5	Dirk Luchte		152	
6	Art Braunschweiger		51	
7	Manuel Pereira		27	
8	Rita Müller		140	
9	José Pedro Freyre		134	
10	Anne Heikkinen		52	
11	Eliane Noz		6	
12	Sven Petersen		33	
13	Martin Bein		39	
14	Lars Peterson		43	
15	Felipe Izquierdo		46	
16	Janete Limeira		114	
17	Martin Sommer		78	
18	Pedro Afonso		125	

Operaciones de conjunto en SQL INTERSECCIÓN

Listar *id_localidad* de clientes y proveedores que residan en la misma localidad

<i>cliente</i>	
PK	<i>id_cliente</i>
FK	<i>id_localidad</i>
FK	<i>id_empresa_tipo</i>
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

<i>proveedor</i>	
PK	<i>id_proveedor</i>
FK	<i>id_localidad</i>
FK	<i>id_empresa_tipo</i>
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

Operaciones de conjunto en SQL INTERSECCIÓN

Listar *id_localidad* de clientes y proveedores que residan en la misma localidad

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select id_localidad
from main.cliente)

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select id_localidad
from main.proveedor)

```
1 select id_localidad
2 from main.cliente
3
```

```
1 select id_localidad
2 from main.proveedor
3
```

Data Output Explain Messages

id_localidad
integer

56	133
57	145
58	147
59	151
60	26
61	32
62	42
63	7
64	2
65	8
66	10
67	12
68	14
69	64
70	40
71	91
72	16
73	76

Data Output Explain Messages Notifications

id_localidad
integer

12	39
13	
14	105
15	
16	11
18	100
19	18
20	143
21	75
22	152
23	52
24	63
25	116
26	89
27	6
28	95
29	91

Compatibilidad de Unión
Igual grado y dominio
columna a columna

Operaciones de conjunto en SQL INTERSECCIÓN

Listar nombre de contacto y id_localidad de clientes y proveedores que residan en la misma localidad

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select nombre_contacto, id_localidad
from main.cliente)

INTERSECT

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select nombre_contacto, id_localidad
from main.proveedor)

```
1 select id_localidad
2 from main.cliente
3
```

Data Output Explain Messages

id_localidad	
integer	
56	133
57	145
58	147
59	151
60	26
61	32
62	42
63	7
64	2
65	8
66	10
67	12
68	14
69	64
70	40
71	91
72	16
73	76

```
1 select id_localidad
2 from main.proveedor
3
```

Data Output Explain Messages Notifications

id_localidad	
integer	
12	39
13	33
14	105
15	120
16	11
17	141
18	100
19	18
20	143
21	75
22	152
23	52
24	63
25	116
26	89
27	6
28	95
29	91

Operaciones de conjunto en SQL INTERSECCIÓN

Listar nombre de contacto y id_localidad de clientes y proveedores que residan en la misma localidad

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select nombre_contacto, id_localidad
from main.cliente)

INTERSECT

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select nombre_contacto, id_localidad
from main.proveedor)

```
1 select id_localidad
2 from main.cliente
3 intersect
4 select id_localidad
5 from main.proveedor
6
```

Data Output	Explain	Messages	Notifications				
<table> <tr> <th>id_localidad</th> <th>integer</th> </tr> <tr> <td>1</td> <td>91</td> </tr> </table>	id_localidad	integer	1	91			
id_localidad	integer						
1	91						

Sólo en la localidad con id =
91 residen clientes y
proveedores

Operaciones de conjunto en SQL DIFERENCIA

Listar *id_localidad* donde sólo residan clientes pero no residan proveedores

```
1 select id_localidad
2 from main.cliente
3
```

```
1 select id_localidad
2 from main.proveedor
3
```

Data Output Explain Messages

id_localidad
integer

56	133
57	145
58	147
59	151
60	26
61	32
62	42
63	7
64	2
65	18
66	10
67	12
68	14
69	64
70	40
71	91
72	16
73	76

Data Output Explain Messages Notifications

id_localidad
integer

12	39
13	
14	105
15	
16	11
18	100
19	18
20	143
21	75
22	152
23	52
24	63
25	116
26	89
27	6
28	95
29	91

Compatibilidad de Unión
Igual grado y dominio
columna a columna

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select id_localidad
from main.cliente)

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select id_localidad
from main.proveedor)

Operaciones de conjunto en SQL DIFERENCIA

Listar *id_localidad* donde sólo residan clientes pero no residan proveedores

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select id_localidad
from main.cliente)

MINUS (EXCEPT)

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select id_localidad
from main.proveedor)

```
1 (select id_localidad
2   from main.cliente)
3 except
4 (select id_localidad
5   from main.proveedor )
6
```

Data Output		Explain	Messages	Notifications
	id_localidad integer			
1	87			
2	74			
3	54			
4	4			
5	34			
6	51			
7	70			
8	146			
9	83			
10	10			
11	45			
12	107			
13	134			
14	86			
15	144			
16	92			
17	93			
18	101			

Operaciones de conjunto en SQL DIFERENCIA

Listar *id_localidad* donde sólo residan clientes pero no residan proveedores

cliente	
PK	id_cliente
FK	id_localidad
FK	id_empresa_tipo
A	nombre_empresa
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax
A	fecha_alta

(select id_localidad
from main.cliente)

MINUS (EXCEPT)

proveedor	
PK	id_proveedor
FK	id_localidad
FK	id_empresa_tipo
A	nombre_proveedor
A	nombre_contacto
A	cargo_contacto
A	direccion
A	telefono
A	fax

(select id_localidad
from main.proveedor)

```
1 (select id_localidad
2   from main.cliente)
3 except
4 (select id_localidad
5   from main.proveedor )
6
```

Data Output		Explain	Messages	Notifications
	id_localidad	integer		
1	87			
2	74			
3	54			
4	4			
5	34			
6	51			
7	70			
8	146			
9	83			
10	10			
11	45			
12	107			
13	134			
14	86			
15	144			
16	92			
17	93			
18	101			

Id_localidades donde
sólo residen clientes y
no residen
proveedores

Ejercitamos con la guía de laboratorio 2 (Ejercicios 11-17)

Funciones de agregación y GROUP By en SQL

SQL proporciona **5 funciones de agregación** que pueden ser empleadas en la cláusula SELECT. Estas funciones son:



- mínimo: min
- promedio: avg
- máximo: max
- total: sum
- cuenta: count

Se llaman **funciones agregación** porque se emplean para operar sobre **grupos de tuplas**. Devuelven un único valor para el grupo de tuplas.

La función **COUNT** devuelve el número de tuplas o valores especificados en una consulta. Las funciones **SUM, MAX, MIN y AVG** se aplican a un conjunto de valores numéricos y devuelven, respectivamente, la suma, el valor máximo, el valor mínimo y el promedio de esos valores.

Las funciones **MAX** y **MIN** también se pueden utilizar con atributos que tienen dominios no numéricos si los valores del dominio tienen una *ordenación total* entre sí. La *ordenación total* se refiere a que para dos valores cualesquiera del dominio, puede determinarse que uno aparece antes que el otro en el orden definido; por ejemplo, los dominios DATE, TIME y TIMESTAMP y las cadenas alfanuméricas.

Sea la siguiente consulta:

Listar la **cantidad** de productos ofrecidos por la empresa, el **total** de unidades en existencias, el **promedio** de los precios unitarios, el **mínimo** nombre de producto y la **máxima** cantidad por unidad de producto.

Listar la cantidad de productos ofrecidos por la empresa, el total de unidades en existencias, el promedio de los precios unitarios, el mínimo nombre de producto y la máxima cantidad por unidad de producto

```
CREATE TABLE main.producto
(
  id_producto integer,
  nombre_producto character varying(40),
  id_categoria integer,
  cantidad_por_unidad character varying(20),
  precio_unidad numeric(6,2),
  unidades_en_existencia smallint,
  unidades_en_pedido smallint,
  nivel_nuevo_pedido smallint,
  suspendido boolean,
  id_proveedor integer,
  CONSTRAINT pk_producto PRIMARY KEY (id_producto),
  CONSTRAINT fk_producto_categoria FOREIGN KEY
(id_categoria)
  REFERENCES main.categoria (id_categoria)
  CONSTRAINT fk_producto_proveedor FOREIGN KEY
(id_proveedor)
  REFERENCES main.proveedor (id_proveedor))
```

```
SELECT
  COUNT(id_producto),
  SUM(unidades_en_existencia),
  AVG(precio_unidad),
  Min(nombre_producto),
  Max(cantidad_por_unidad)
FROM main.producto;
```

Query Editor

Query History

```
1 SELECT
2 COUNT(id_producto),
3 SUM(unidades_en_existencia),
4 AVG(precio_unidad),
5 Min(nombre_producto),
6 Max(cantidad_por_unidad)
7 FROM main.producto;
8
```

Data Output

Messages

Notifications

	count bigint	sum bigint	avg numeric	min text	max text
1	77	3119	28.8663636363636364	Algas Konbu	paq. 5 kg

```

SELECT
    COUNT(id_producto),
    SUM(unidades_en_existencia),
    AVG(precio_unidad),
    Min(nombre_producto),
    Max( cantidad_por_unidad)
FROM main.producto;

```

Como se observa en el ejemplo, las funciones de agregación se aplicaron sobre todas las tuplas de la relación Producto, obteniendo para cada función un único valor, por ej. COUNT muestra la cantidad de productos en la tabla Producto. En este caso ***se tiene un único grupo, que está integrado por todas las tuplas de la relación Producto.***

Query Editor

Query History

1

SELECT

2

COUNT(id_producto),

3

SUM(unidades_en_existencia),

4

AVG(precio_unidad),

5

Min(nombre_producto),

6

Max(cantidad_por_unidad)

7

FROM main.producto;

8

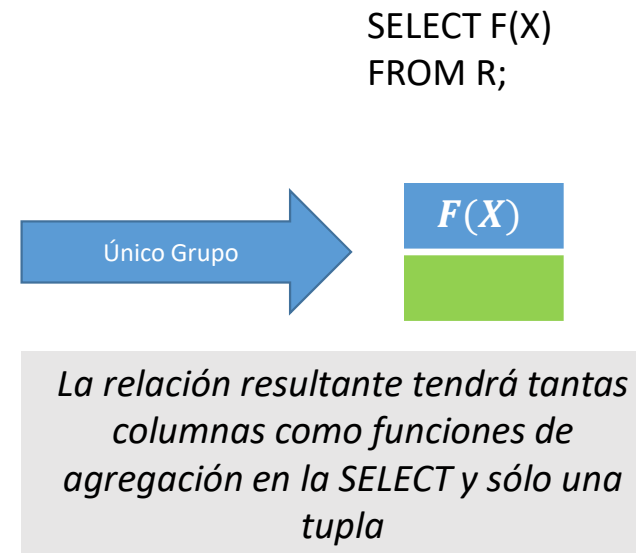
Data Output

Messages

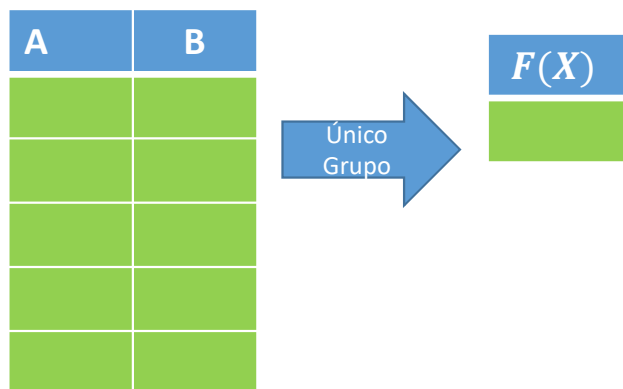
Notifications

	count bigint	sum bigint	avg numeric	min text	max text
1	77	3119	28.8663636363636364	Algas Konbu	paq. 5 kg

R	
A	B



Funciones de agregación y GROUP By en SQL



Las funciones de agregación se aplican a toda la tabla, y son las únicas columnas a proyectar en la cláusula SELECT.

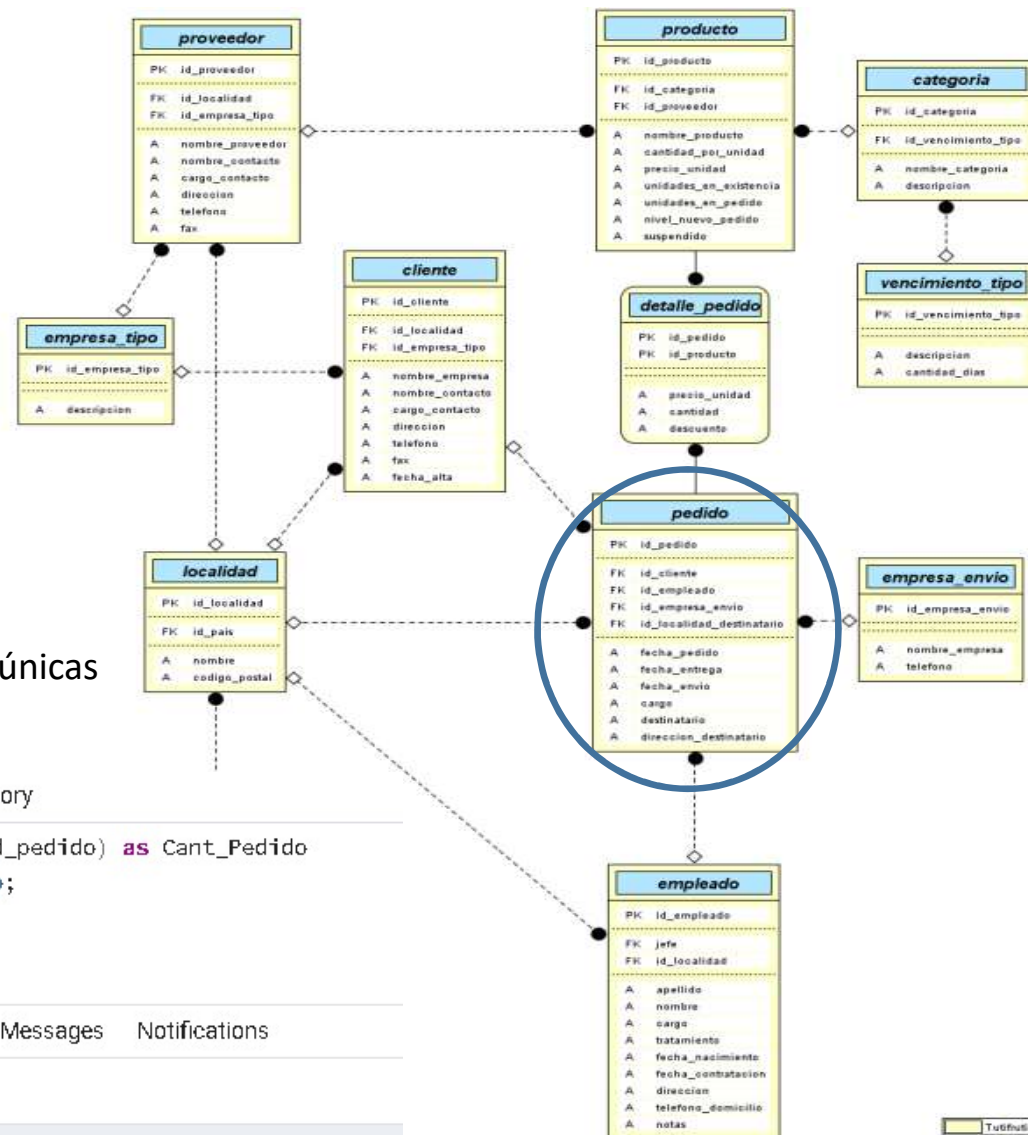
Listar la cantidad de pedidos

```
SELECT count(id_pedido) as Cant_Pedido
FROM main.pedido;
```

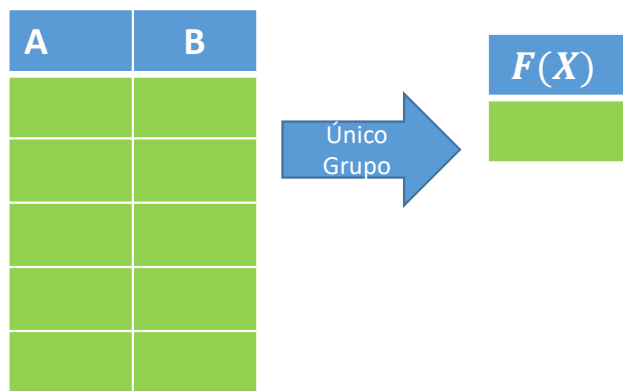
Query Editor		Query History	
1	SELECT	count(id_pedido)	as Cant_Pedido
2	FROM	main.pedido;	
3			
4			

Data Output		Explain	Messages	Notifications
	cant_pedido			
	bigint			
1	830			

Un grupo (la tabla)



Funciones de agregación y GROUP By en SQL

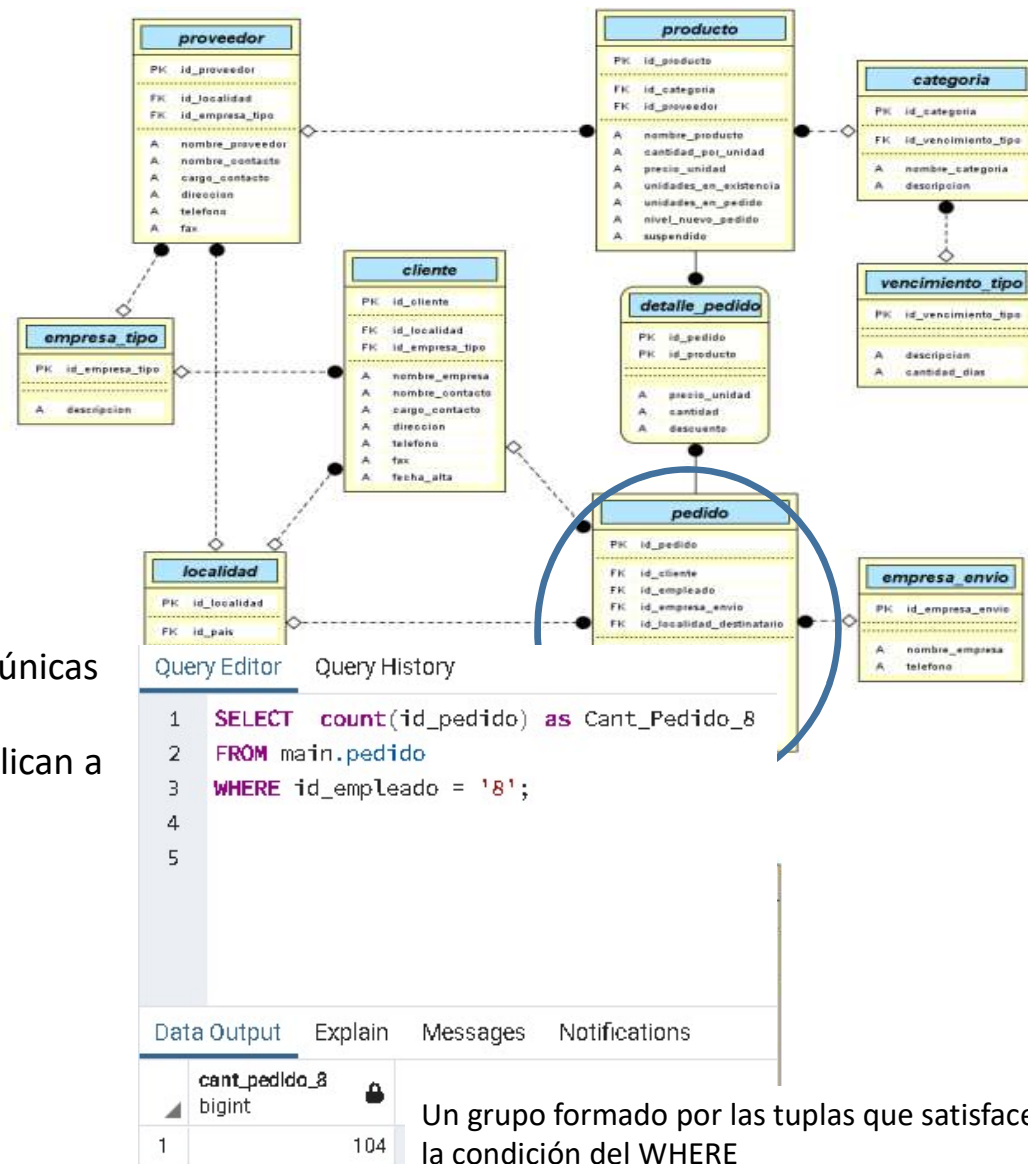


Las funciones de agregación se aplican a toda la tabla, y son las únicas columnas a proyectar en la cláusula SELECT.

Si se tiene la cláusula WHERE, las funciones de agregación se aplican a las tuplas que satisfacen la misma.

Listar la cantidad de pedidos del empleado 8.

```
SELECT count(id_pedido) as Cant_Pedido
FROM main.pedido
WHERE id_empleado = '8';
```



Funciones de agregación y GROUP By en SQL

El asterisco (*) se refiere a las filas (tuplas), por lo que COUNT (*) devuelve el número de filas del grupo como resultado de la consulta.

Listar la cantidad de pedidos del empleado 8.

```
SELECT count(*) as Cant_Pedido
FROM main.pedido
WHERE id_empleado = '8';
```

Query Editor		Query History	
1	SELECT count(id_pedido) as Cant_Pedido_8		
2	FROM main.pedido		
3	WHERE id_empleado = '8';		
4			
5			

Data Output		Explain	Messages	Notifications
	cant_pedido_8 bigint			
1	104			



Query Editor		Query History	
1	SELECT count(*) as Cant_Pedido_8		
2	FROM main.pedido		
3	WHERE id_empleado = '8';		
4			
5			

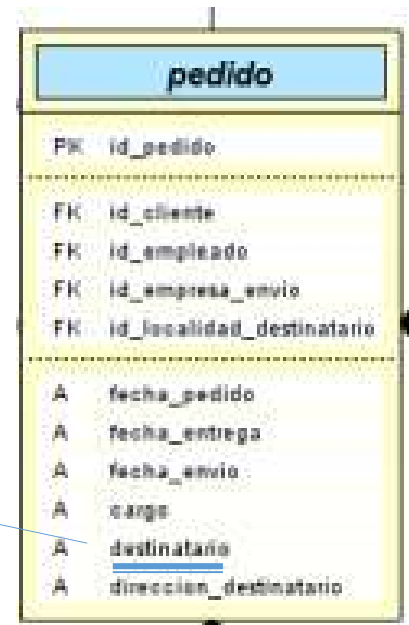
Data Output		Explain	Messages	Notifications
	cant_pedido_8 bigint			
1	104			

Funciones de agregación y GROUP By en SQL

Podemos utilizar la función COUNT para contar los valores en una columna.

Listar la cantidad de destinatarios distintos en los pedidos.

```
SELECT count(distinct destinatario)
FROM main.pedido;
```



Query Editor		Query History	
1	<pre>select count (destinatario) from main.pedido</pre>		
2			
3			
4			
Data Output		Explain	Messages
		Notifications	
	count bigint 		
1	830		

Cuenta todos los valores en la columna

Cuenta todos los valores en la columna solicitada.



Query Editor

Query History

1

2

3

4

```
select count (distinct destinatario) from main.pedido
```

Data Output

Explain

Messages

Notifications

count
bigint

1

89

Sólo cuenta valores distintos en la columna

Sólo cuenta valores distintos en la columna solicitada, eliminando duplicados.

Funciones de agregación y GROUP By en SQL

Los valores NULL se descartan cuando se aplican las funciones agregadas a una columna en particular

Listar la cantidad de proveedores en los productos comercializados.

```
SELECT count(distinct id_proveedor)
FROM main.producto;
```

Query Editor		Query History	
1	select id_proveedor		
2	from main.producto		
3			
Data Output		Explain	Messages
Id_proveedor			
integer			
1	1		
2	1		
3	12		
4	7		
5	3		
6	3		
7	2		
8	2		
9	2		
10	2		
11	1		
12	[null]		
13	11		
14	11		

producto	
PK:	id_producto
FK:	id_categoria
FK:	id_proveedor
A:	nombre_producto
A:	cantidad_por_unidad
A:	precio_unidad
A:	unidades_en_existencia
A:	unidades_en_pedido
A:	nivel_nuevo_pedido
A:	suspendido

Query Editor		Query History	
1	select count(id_proveedor)		
2	from main.producto		
3			
4			
Data Output		Explain	Messages
count			
bigint			
1	76		

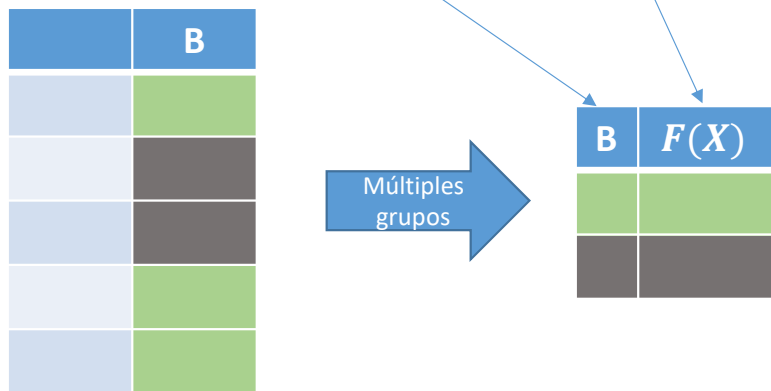
No contó la tupla con NULL en la columna considerada

Funciones de agregación y GROUP BY en SQL

Listar la cantidad de pedidos realizados por cada empleado

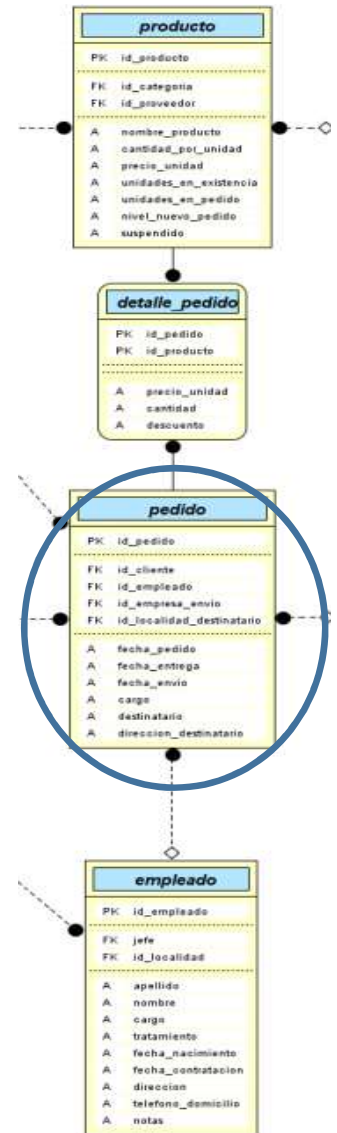
```
SELECT p.id_empleado, count(id_pedido) as Cant_Pedido  
FROM main.pedido p  
GROUP BY p.id_empleado
```

La cláusula GROUP BY crea los grupos, en este caso un grupo por cada id_empleado.



La tabla resultado tendrá una columna por cada función de agregación, y/o columnas de la cláusula GROUP BY y tendrá una tupla por cada grupo. Las columnas que se encuentran en el GROUP BY podrían no ser proyectadas en la SELECT

Las funciones de agregación se pueden aplicar a grupos de tuplas, dentro del conjunto de datos generado en el FROM. Para ello, utiliza la cláusula GROUP BY seguida de la(s) columna(s) por las cuales se forman los grupos de tuplas a los cuales se aplican las funciones de agregación.



Funciones de agregación y GROUP By en SQL

Listar la cantidad de pedidos realizados por cada empleado

```
SELECT p.id_empleado, count(id_pedido) as Cant_Pedido
FROM main.pedido p
GROUP BY p.id_empleado
```

Query Editor		Query History	
1	SELECT	p.id_empleado,	count(id_pedido) as Cant_Pedido
2	FROM	main.pedido p	
3	GROUP BY	p.id_empleado	
4			
5			

Data Output		Explain	Messages	Notifications
	Id_empleado integer		cant_pedido bigint	
1	8		104	Un valor de la función (COUNT) por cada grupo (id_empleado)
2	9		43	
3	7		72	
4	1		123	
5	5		42	
6	4		156	
7	2		96	
8	6		67	
9	3		127	

Funciones de agregación y GROUP By en SQL

Listar la cantidad de pedidos realizados por cada empleado

```
SELECT p.id_empleado, count(id_pedido) as Cant_Pedido
FROM main.pedido p
GROUP BY p.id_empleado
```

Query Editor		Query History	
1	SELECT	p.id_empleado	
2	FROM	main.pedido p	
3			

Data Output		Explain	Messages	Notifications
	Id_empleado integer			
1	1			
2	8			
3	2			
4	1			
5	4			
6	2			
7	7			
8	8			
9	4			
10	1			
11	6			
12	1			
13	4			
14	6			
15	7			
16	8			
17	9			
18	2			

Query Editor		Query History	
1	SELECT	p.id_empleado, count(id_pedido) as Cant_Pedido	
2	FROM	main.pedido p	
3	GROUP BY	p.id_empleado	
4			

Data Output		Explain	Messages	Notifications
	Id_empleado integer		cant_pedido bigint	
1	8		104	
2	9		43	
3	7		72	
4	1		123	
5	5		42	
6	4		156	
7	2		96	
8	6		67	
9	3		127	

Un valor de la función por cada grupo (empleado). Resumen la colección de tuplas para cada empleado.

Cada grupo estará compuesto por las tuplas que tienen el mismo valor para algún(os) atributo(s), denominado(s) atributo(s) de agrupamiento y son las que se encuentran en la cláusula GROUP BY

Funciones de agregación y GROUP By en SQL

Listar la cantidad de pedidos realizados por cada empleado

```
SELECT p.id_empleado, count(id_pedido) as Cant_Pedido
FROM main.pedido p
GROUP BY p.id_empleado
```



Las columnas que se pueden proyectar en la cláusula SELECT son las funciones de agregación y opcionalmente sólo las columnas que se encuentran en la cláusula GROUP BY.



```
SELECT count(id_pedido) as Cant_Pedido
FROM main.pedido p
GROUP BY p.id_empleado
```

Query Editor		Query History	
1	SELECT	p.id_empleado,	count(id_pedido) as Cant_Pedido
2	FROM	main.pedido p	
3	GROUP BY	p.id_empleado	
4			
5			

Data Output		Explain	Messages	Notifications
	Id_empleado integer		cant_pedido bigint	
1			8	104
2			9	43
3			7	72
4			1	123
5			5	42
6			4	156
7			2	96
8			6	67
9			3	127

Columnas
proyectadas en la
cláusula SELECT

Un valor de la
función por
cada grupo
(empleado)

Query Editor		Query History	
1	SELECT	count(id_pedido)	as Cant_Pedido
2	FROM	main.pedido p	
3	GROUP BY	p.id_empleado	
4			
5			

Data Output		Explain	Messages	Notifications
	cant_pedido bigint			
1			104	
2			43	
3			72	
4			123	
5			42	
6			156	
7			96	
8			67	
9			127	

Un valor de la
función por
cada grupo
(empleado)

Funciones de agregación y GROUP By en SQL

Si existen NULLs en el atributo de agrupamiento, se crea un grupo separado para todas las tuplas con un valor NULL en el atributo de agrupamiento.

Listar la cantidad de productos distintos por proveedor

```
SELECT id_proveedor, count(id_producto)
FROM main.producto
GROUP BY p.id_proveedor
```

```
1 SELECT id_proveedor, count(id_producto)
2 FROM main.producto
3 GROUP BY id_proveedor
4
```

	Data Output	Explain	Messages	Notifications
	id_proveedor integer	count bigint		
1		8	4	
2		11	3	
3		[null]	1	
4		29	2	
5		4	3	
6		10	1	
7		9	2	
8		7	5	
9		15	3	
10		6	2	
11		26	2	
12		12	5	
13		24	3	
14		19	2	
15		25	2	
16		21	2	
17		14	3	
18		3	3	
19		17	3	
20		28	2	

Grupo NULL

producto	
FK	id_producto
FK	id_categoria
FK	id_proveedor
A	nombre_producto
A	cantidad_por_unidad
A	precio_unidad
A	unidades_en_existencia
A	unidades_en_pedido
A	nivel_nuevo_pedido
A	suspendido

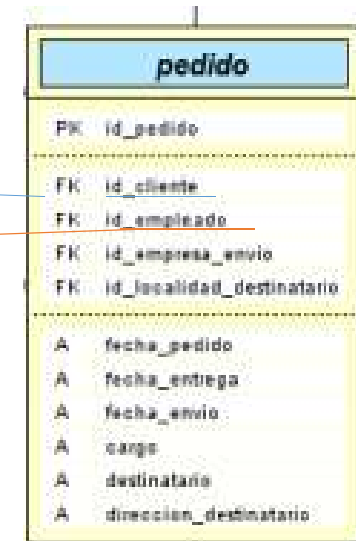
Funciones de agregación y GROUP By en SQL

La **retención de duplicados** es importante al aplicar algunas de las funciones de agregación.

Si **queremos eliminar duplicados**, usamos la palabra clave **DISTINCT** en la expresión de agregados.

“Listar cantidad de distintos cliente atendidos por cada”.

```
select id_empleado, count (distinct id_cliente)
from main.pedido
group by p.id_empleado;
```



pedido	
PK	id_pedido
FK	<u>id_cliente</u>
FK	<u>id_empleado</u>
FK	id_empresa_envio
FK	id_localidad_destinatario
A	fecha_pedido
A	fecha_entrega
A	fecha_envio
A	cargo
A	destinatario
A	direccion_destinatario

Funciones de agregación y GROUP By en SQL

La **retención de duplicados** es importante al aplicar algunas de las funciones de agregación.

Si **queremos eliminar duplicados**, usamos la palabra clave **distinct** en la expresión de agregados.

“Listar cantidad de distintos cliente atendidos por cada ”.

```
select id_empleado, count (distinct id_cliente)
from main.pedido
group by p.id_empleado;
```

pedido	
PK	id_pedido
FK	id_cliente
FK	id_empleado
FK	id_empresa_envio
FK	id_localidad_destinatario
A	fecha_pedido
A	fecha_entrega

Query Editor

Query History

```

1 select p.id_empleado, count (p.id_cliente)
2 from main.pedido p
3 group by p.id_empleado;
4

```

Data Output

Explain

Messages

Notifications

	id_empleado integer	count bigint	
1	8	104	
2	9	43	
3	7	72	
4	1	123	
5	5	42	
6	4	156	
7	2	96	
8	6	67	
9	3	127	

Con repetidos

Query Editor
Query History

```

1  select p.id_empleado, count (distinct p.id_cliente)
2  from main.pedido p
3  group by p.id_empleado;
4

```

Data Output
Explain
Messages
Notifications

	id_empleado integer	count bigint
1	1	65
2	2	59
3	3	63
4	4	75
5	5	28
6	6	43
7	7	45
8	8	56
9	9	29

Sin repetidos

Funciones de agregación y GROUP By en SQL

Para seleccionar grupos que cumplan con una dada condición se utiliza la cláusula HAVING, seguido a un agrupamiento (GROUP BY).

“Listar los empleados que han realizado más de 100 pedidos”

```
SELECT id_empleado, count (id_pedido)
FROM main.pedido
GROUP BY id_empleado
HAVING count (id_pedido) > 100
```

Query Editor Query History

```

1 select id_empleado, count (id_pedido)
2 from main.pedido
3 group by id_empleado;
4
5

```

Data Output Explain Messages Notifications

	Id_empleado integer	count bigint
1	8	104
2	9	43
3	7	72
4	1	123
5	5	42
6	4	156
7	2	96
8	6	67
9	3	127

Todos los grupos de empleado

Todos los grupos
de empleado

Query Editor

```

1 select id_empleado, count (id_pedido)
2 from main.pedido
3 group by id empleado
4 having count (id_pedido) > 100
5

```

Data Output

	Id_empleado integer	count bigint
1	8	104
2	1	123
3	4	156
4	3	127

Explain

Messages

Notifications

Grupos que s
condición d

Grupos que satisfacen la condición del HAVING



Funciones de agregación y GROUP By en SQL

Si en la **misma consulta** aparecen una cláusula **WHERE** y una cláusula **HAVING**, **primero** se aplica el predicado de la **cláusula WHERE**. Las tuplas **que satisfacen** el predicado **WHERE son colocadas en grupos** por la cláusula **GROUP BY**. **Después se aplica la cláusula HAVING** a cada grupo. Los grupos que satisfacen el predicado de la cláusula **HAVING** son utilizados por la cláusula **SELECT** para generar tuplas del resultado de la consulta.

“Listar los empleados con id iguales a 1,4 o 5, que hubieran efectuado más de 100 pedidos.”

```
select id_empleado, count(id_pedido)
from main.pedido
where id_empleado = '1' or id_empleado = '4' or id_empleado = '5'
group by id_empleado
having count (id_pedido) > 100
```

Query Editor

Query History

```

1  select id_empleado, count(id_pedido)
2  from main.pedido
3  where id_empleado = '1' or id_empleado = '4' or id_empleado = '5'
4  group by id_empleado
5  having count (id_pedido) > 100;

```

Data Output

Explain

Messages

Notifications

	Id_empleado integer	count bigint	
1	1	123	Grupos que satisfacen la condición del HAVING.
2	4	156	

Query Editor Query History

```
1 select id_empleado, id_pedido
2 from main.pedido
3 where id_empleado = '1' or id_empleado = '4' or id_empleado = '5';
4
5
```

Data Output Explain Messages Notifications

	Id_empleado integer	Id_pedido [PK] integer
1	1	11069
2	1	11071
3	4	11072
4	4	11076
5	1	11077
6	1	11039
7	4	11040
8	4	11061
9	4	11062
10	5	10248
11	4	10250
12	4	10252
13	5	10254
14	4	10257
15	1	10258
16	5	10841
17	1	11038
18	5	11043

1.- Obtiene la tuplas que satisfacen el WHERE

Query Editor Query History

```
1 select id_empleado, count(id_pedido)
2 from main.pedido
3 where id_empleado = '1' or id_empleado = '4' or id_empleado = '5'
4 group by id_empleado;
5
```

Data Output Explain Messages Notifications

	Id_empleado integer	count bigint
1	1	123
2	5	42
3	4	156

2.- Realiza los grupos con las tuplas que satisfacen el WHERE

Query Editor Query History

```
1 select id_empleado, count(id_pedido)
2 from main.pedido
3 where id_empleado = '1' or id_empleado = '4' or id_empleado = '5'
4 group by id_empleado
5 having count(id_pedido) > 100;
```

Data Output Explain Messages Notifications

	Id_empleado integer	count bigint
1	1	123
2	4	156

3.- Selecciona los grupos que satisfacen la cláusula HAVING

Funciones de agregación y GROUP By en SQL

Una consulta con cláusula GROUP BY con o sin HAVING, puede solicitar utilizar la cláusula ORDER BY para que la proyección de columna tenga un criterio de ordenación.

“Listar por orden descentente los empleados con id menores a 5, que hubieran efectuado más de 100 pedidos,”

```
select id_empleado, count(id_pedido)
from main.pedido
where id_empleado = '1' or id_empleado = '4' or id_empleado = '5'
group by id_empleado
having count (id_pedido) > 100
Order by id_empleado DESC;
```

```
Query Editor Query History
```

```
1 select id_empleado, count(id_pedido)
2 from main.pedido
3 where id_empleado = '1' or id_empleado = '4' or id_empleado = '5'
4 group by id_empleado
5 having count (id_pedido) > 100
6 Order by id_empleado DESC;
```

```
Data Output Explain Messages Notifications
```

	Id_empleado integer	count bigint	
1	4	156	} Grupos que satisfacen la condición del H ordenados de forma descendente
2	1	123	

Funciones de agregación y GROUP BY en SQL

Una consulta con cláusula GROUP BY con o sin HAVING, puede solicitar utilizar la cláusula ORDER BY para que la proyección de columna tenga un criterio de ordenación.

“Listar por orden ascendente los empleados con la cantidad de pedidos efectuados”

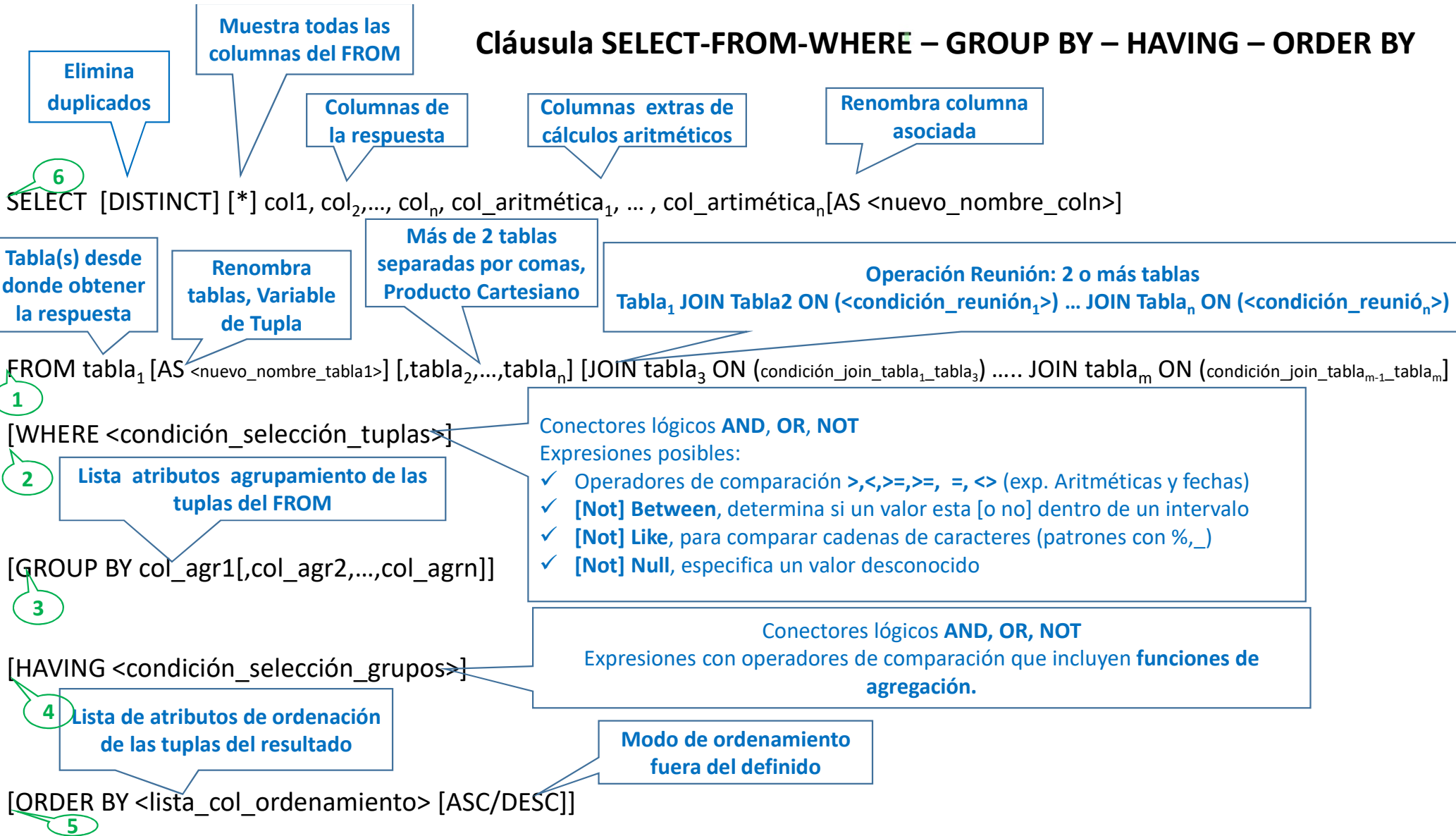
```
select id_empleado, count (distinct id_cliente)
from main.pedido
group by id_empleado
Order by id_empleado;
```

Query Editor		Query History	
1	select id_empleado, count (distinct id_cliente)		
2	from main.pedido		
3	group by id_empleado		
4	Order by id_empleado;		
5			

Data Output				Explain	Messages	Notifications
	id_empleado integer		count bigint			
1		1	65			
2		2	59			
3		3	63			
4		4	75			
5		5	28			
6		6	43			
7		7	45			
8		8	56			
9		9	29			

Grupos que satisfacen la condición del HAVING, ordenados de forma ascendente.

Cláusula SELECT-FROM-WHERE – GROUP BY – HAVING – ORDER BY



Ejercitamos con la guía de laboratorio 2 (ejercicios 26 a 29)